Job Automation Agent…

Problem Statement: Autonomous Job Automation Agent

The proposed system is an autonomous job automation agent that can manage most of the job search and application process on behalf of a candidate.

The candidate provides a Master Resume and basic preferences such as up to 3 target job titles, preferred locations, experience level, salary expectations, and work preferences. The agent then searches for suitable jobs, customizes applications, communicates with recruiters, tracks responses, and continuously improves its approach based on previous results.

1. Automated Job Search, Resume Customization and Job Applications

The system should automatically search for the latest relevant jobs across major job portals such as LinkedIn, Naukri, Indeed, Glassdoor, Foundit, Wellfound, Internshala, Cutshort, Instahyre, Hirist, Shine, TimesJobs, company career portals, and other supported job platforms.

The agent should identify jobs based on the candidate's selected job titles and profile.

For every job, it should analyze the Job Description against the Master Resume and determine the relevance of the opportunity based on skills, experience, job title, technologies, education, projects, location, work mode, and other job requirements.

For relevant jobs, the agent should automatically create a job-specific ATS-friendly resume.

The resume should be generated using Overleaf and LaTeX, with relevant skills, experience, projects, and keywords emphasized according to the Job Description.

The system should then use the customized resume to complete the application process.

If the application contains questions, the agent should answer them using the candidate's verified profile and Master Resume.

The system should also maintain permissions for each job portal so the candidate can control which platforms the agent is allowed to access and apply through.

A complete history of every application should be maintained, including the job, company, portal, resume version, application date, application status, questions answered, and application result.

2. HR Contact Discovery and Automated Cold Email Outreach

The second major functionality is an automated recruiter and HR outreach system.

The agent should search publicly available sources to identify relevant HR professionals, recruiters, talent acquisition specialists, hiring managers, and recruitment contacts.

The system should connect these contacts with relevant companies and job opportunities.

For suitable opportunities, the agent should automatically create a personalized cold email containing a short personalized introduction, relevant candidate experience, reason for contacting the recruiter, job or role reference, customized ATS-friendly resume, job-specific cover letter, and relevant portfolio or professional links.

The email should be natural, concise, personalized, and human-like rather than looking like a generic generated template.

The system should also track emails sent, delivery status, replies, positive responses, negative responses, follow-ups, and no-response cases.

The agent should optionally send follow-up emails when no response is received within a configured period.

3. Self-Learning and Continuous Improvement

The third major capability is the agent's ability to learn from previous applications and communication outcomes.

The system should analyze historical job applications, customized resumes, cover letters, cold emails, recruiter responses, rejections, interview invitations, screening results, and no-response cases.

Based on these results, the agent should continuously improve future applications.

For example, it should identify which resume formats perform better, which skills and keywords are frequently relevant, which email formats receive more responses, which subject lines generate better engagement, which cover-letter structures work better, which application questions frequently appear, which answers are appropriate for recurring questions, and which types of jobs generate more positive responses.

The agent should use this information to improve future resumes, cover letters, emails, and application answers without changing factual information about the candidate.

The system should maintain a Candidate Knowledge Base containing verified information that can be safely reused when answering application questions.

For uncertain or sensitive questions, the agent should be able to request user confirmation rather than making up information.

4. Job Search Dashboard, Analytics and Activity Tracking

The system should provide a centralized dashboard where the candidate can monitor the entire job-search process.

The dashboard should display job statistics such as total jobs discovered, relevant jobs, jobs applied to, jobs by portal, jobs by company, jobs by job title, and jobs applied today, this week, and this month.

The dashboard should display application statistics such as applications submitted, applications in progress, applications viewed, recruiter responses, interview invitations, rejections, no-response applications, and offers.

The dashboard should also display email and outreach statistics such as HR contacts discovered, emails sent, emails replied to, positive responses, negative responses, follow-ups sent, and pending follow-ups.

The system should analyze received emails and categorize them into meaningful groups such as positive, negative, neutral, interview opportunity, rejection, follow-up required, and action required.

The dashboard should also provide a complete timeline of what the agent has done.

For example, a typical activity could be:

Job Found

Job Analyzed

Resume Customized

Application Submitted

HR Contact Found

Cold Email Sent

Recruiter Response Received

Response Analyzed

Interview, Rejection, or Follow-up

This allows the candidate to understand exactly what happened during the day and what actions require attention.

Additional Important Capabilities

Human Approval Controls

The candidate should be able to choose between different operating modes.

In Fully Autonomous Mode, the agent performs approved actions automatically.

In Approval Mode, the agent prepares applications and emails but asks for approval before sending or submitting them.

In Assisted Mode, the agent performs research and preparation while the candidate performs the final submission.

Duplicate Prevention

The system should detect when the same job appears on multiple portals and prevent duplicate applications.

Application Limits

The candidate should be able to configure limits such as maximum applications per day, maximum applications per portal, preferred locations, minimum salary, allowed job types, and preferred experience level.

Security and Audit Trail

Every important action should be recorded so the candidate can see what the agent did, which job it interacted with, which resume was used, which email was sent, which questions were answered, when the action happened, and what the result was.

Sensitive credentials and personal information should be securely stored and protected.

Overall Objective

The objective is to build a persistent autonomous career agent that manages the complete job-search workflow.

The complete workflow should cover finding jobs, analyzing jobs, customizing resumes, applying to jobs, finding recruiters, sending personalized emails, tracking responses, analyzing results, learning from previous outcomes, and improving future applications.

Instead of the candidate manually repeating these activities across multiple platforms, the agent should continuously handle the repetitive work while keeping the candidate in control of important decisions and providing complete visibility through the dashboard.

Testing Agent

Problem Statement: Autonomous AI Testing and QA Agent

The proposed system is an autonomous QA and testing agent designed to perform the responsibilities of a senior QA engineer across the complete software application.

The user should only need to provide the application URL or environment, such as localhost, development, QA, staging, UAT, production, or any other deployed environment.

The agent should then understand the application, explore its functionality, create and execute tests, identify defects, validate the application against requirements and designs, and generate a complete QA report.

The system should support web applications, APIs, databases, and mobile applications.

1. Application Understanding and Autonomous Exploration

The first responsibility of the agent should be to understand the application before testing it.

After receiving access to an environment, the agent should automatically explore the application and understand the application structure, navigation, pages and screens, user flows, forms, buttons and actions, authentication, roles and permissions, API interactions, data flows, important business workflows, error states, loading states, empty states, and responsive behavior.

The agent should behave similar to a senior QA engineer performing exploratory testing.

Instead of only testing predefined scenarios, it should actively explore the application and identify areas that may contain defects.

It should maintain an internal understanding of the relationship between pages, components, actions, APIs, databases, and expected results.

This application knowledge should then be reused while creating subsequent test cases.

2. Automated Test Case Generation and Playwright Execution

The agent should automatically create test cases based on its understanding of the application.

It should generate and execute Playwright tests for web applications.

The generated tests should cover smoke testing, functional testing, regression testing, end-to-end testing, negative testing, form validation, navigation testing, authentication, authorization, role-based access, error handling, boundary conditions, and responsive behavior.

For example, a complete workflow could include login, dashboard navigation, creating a product, validating the product, editing the product, deleting the product, and verifying the database.

The agent should not simply generate tests randomly.

It should identify important business workflows and prioritize them based on application behavior and risk.

The system should automatically generate test files, execute tests, capture failures, capture screenshots, capture videos where required, capture traces, capture console errors, capture network failures, record execution time, analyze failures, and generate reports.

3. UI, Visual and Figma Validation

The agent should also perform visual QA, similar to how a manual senior QA engineer compares an implementation against the intended design.

The system should support Figma designs, screenshots, design references, and UI specifications.

The agent should compare the actual application UI against the expected design.

It should identify issues such as incorrect spacing, alignment problems, incorrect font sizes, wrong colors, missing elements, incorrect component sizes, incorrect button states, layout issues, responsive problems, overflow, text wrapping, missing icons, incorrect images, modal and dialog issues, table alignment, and mobile UI problems.

The agent should produce visual evidence using screenshots and clearly identify where the implementation differs from the expected design.

4. Browser DevTools and Runtime Investigation

The agent should be capable of performing deeper browser-level investigation rather than only looking at the rendered UI.

It should analyze information available through browser developer tools, including console errors, console warnings, network requests, failed API calls, HTTP status codes, request and response payloads, request timing, resource loading, JavaScript errors, browser storage, cookies, local storage, session storage, and performance information.

The agent should correlate these findings with visible UI issues.

For example, if a UI action fails, the agent should investigate whether it is caused by a failed API request, server error, backend exception, or database issue.

Instead of reporting only that a button is not working, the agent should attempt to identify the underlying technical reason.

5. Accessibility Testing

The agent should perform automated and exploratory accessibility testing.

It should identify issues involving missing labels, incorrect ARIA attributes, keyboard navigation, focus management, focus visibility, heading structure, form accessibility, screen-reader compatibility, color contrast, button accessibility, link accessibility, modal accessibility, table accessibility, error message accessibility, and semantic HTML.

The agent should also verify important accessibility behavior using keyboard interaction and appropriate accessibility tooling.

Accessibility issues should be reported with the affected page, element, issue, severity, evidence, and recommended fix.

6. API Testing

The agent should test the application's backend APIs independently from the UI.

It should automatically discover APIs through available application behavior and network traffic and test GET, POST, PUT, PATCH, DELETE, authentication, authorization, request validation, response validation, status codes, error responses, required fields, invalid inputs, boundary values, duplicate requests, pagination, filtering, sorting, and rate-related behavior where applicable.

The agent should validate both API behavior and the integration between the UI and APIs.

It should verify the complete flow from a UI action to the API request, backend response, UI update, and corresponding database change.

Any inconsistency between these layers should be reported.

7. Database QA and Data Validation

The system should also be capable of testing the application's database layer where authorized database access is provided.

It should validate data creation, data updates, data deletion, data relationships, required fields, data types, duplicate records, missing records, referential integrity, unexpected values, transaction behavior, and API-to-database consistency.

For example, when a user creates an order, the agent should verify that the API returns the correct result, the database record is created correctly, the correct customer is linked, the correct product is linked, and the correct order status is stored.

The agent should identify cases where the UI or API reports success but the underlying database state is incorrect.

8. Performance and Speed Testing

The agent should evaluate application performance and identify slow areas.

It should analyze page load time, API response time, network requests, resource loading, JavaScript execution, rendering performance, large payloads, slow database or API operations, slow pages, and slow user workflows.

It should also support performance and load testing where configured.

The system should identify potential bottlenecks and provide evidence rather than simply reporting that an application is slow.

9. Mobile Application Testing

The same QA agent should support mobile applications.

It should test applications across supported mobile environments and validate installation, launch, login, navigation, forms, API integration, permissions, push notifications, deep links, orientation, different screen sizes, network conditions, error handling, application crashes, and UI consistency.

The agent should perform both functional mobile testing and UI and UX mobile testing where the required environment and device access are available.

10. Intelligent Bug Detection and Root Cause Analysis

The primary objective of the agent should be to find real defects, not simply execute test cases.

When an issue is discovered, the agent should investigate it further.

For each potential bug, it should collect the environment, page or module, steps to reproduce, expected result, actual result, screenshot, console errors, network information, API response, relevant logs where available, test case, reproducibility, severity, priority suggestion, and possible root cause.

The agent should attempt to determine whether the issue is a UI defect, frontend logic defect, API defect, backend defect, database defect, integration defect, performance issue, accessibility issue, security-related issue, or environment and configuration issue.

It should also reduce false positives by attempting to reproduce suspicious failures before reporting them as confirmed defects.

11. Regression Testing and Continuous QA

The agent should maintain a growing regression test suite.

Whenever new functionality is introduced, it should determine which existing tests are affected, which workflows need regression testing, whether existing functionality has broken, and whether new bugs have been introduced.

The system should maintain reusable tests instead of generating completely new tests every time.

Over time, the QA system should build a structured knowledge base of the application containing application knowledge, test cases, test results, bugs, fixes, and regression tests.

12. QA Dashboard and Reporting

The system should provide a centralized QA dashboard containing the complete testing status.

The dashboard should display total test cases, passed tests, failed tests, blocked tests, skipped tests, flaky tests, and regression results.

It should also display bug statistics including total bugs, critical bugs, high bugs, medium bugs, low bugs, open bugs, fixed bugs, reopened bugs, and duplicate bugs.

The dashboard should provide visibility into different quality areas including UI, functional testing, API testing, database testing, accessibility, performance, mobile testing, and security checks.

The dashboard should allow results to be viewed separately for development, QA, staging, UAT, production, and other supported environments.

The system should also provide an execution timeline showing when the application was connected, explored, tested, and analyzed, including test generation, Playwright execution, API testing, database validation, accessibility analysis, performance analysis, visual and Figma comparison, bug investigation, and QA report generation.

13. Self-Learning QA Agent

The agent should continuously improve its testing strategy based on previous testing cycles.

It should learn from previous bugs, previously failed tests, fixed bugs, reopened bugs, frequently failing modules, flaky tests, production issues, developer fixes, and regression failures.

For example, if a particular module repeatedly introduces defects, the agent should automatically increase the depth of testing around that module.

Similarly, when a bug is fixed, the agent should automatically verify that the original bug no longer exists, create a regression test, and verify that related functionality still works.

Overall Objective

The objective is to build an Autonomous Senior QA Agent capable of understanding and testing an entire software product rather than simply executing predefined automated tests.

The complete workflow should cover connecting to the application, understanding the application, exploring the application, generating tests, executing tests, inspecting the UI, comparing the UI with Figma, testing APIs, validating databases, checking accessibility, analyzing performance, testing mobile applications, investigating failures, finding bugs, identifying root causes, generating reports, creating regression tests, and learning from previous results.

The goal is to create a QA agent that can perform a large portion of the day-to-day work normally handled by a senior QA engineer while maintaining complete test evidence, reproducibility, traceability, and visibility through a centralized dashboard.